EVIDENCE-GATED DELIVERY

Orchestrating AI for Secure Software Delivery

A Fortmilo engineering-methodology publication about human authority, evidence gates and multi-model engineering.

Version 1.0 accessible edition - Current

Field	Value
Author	Luca Pacini
Publisher	Fortmilo
Version	1.0 accessible edition
Status	Current
Publication date	30 August 2026
Bounded case study	Security Observatory. It is used only as an engineering case study; this publication makes no claim that Security Observatory contains, offers or depends on AI product capabilities.
Method status	Derived from one bounded project; not independently replicated or validated as a general delivery standard.
Document licence	CC BY 4.0 for Fortmilo-authored text and figures, subject to third-party material and trademark boundaries.

AI may extend delivery capacity; it does not acquire delivery authority.

Abstract

Small software projects make architectural, security, quality and release decisions comparable in consequence to those made by larger teams, but cannot reproduce the same division of labour. Generative artificial intelligence can reduce some of that capacity constraint by accelerating research, drafting, implementation, test construction, documentation and review. Used without an explicit control model, the same capability can amplify error.

This paper presents evidence-gated AI-assisted delivery, a human-governed method for small security-sensitive software projects. Humans retain ownership of intent, security boundaries, public claims and release decisions. AI systems operate within bounded task contracts. Automated checks establish reproducible evidence. Role-separated adversarial challenge exposes assumptions, but model or provider separation is not represented as independent third-party assurance.

The method uses nine linked states: human-owned intent and constraints; a bounded issue contract; AI-assisted implementation; automated proof; role-separated adversarial challenge; human adjudication; a release gate; live verification; and governance close-out. Each transition requires explicit evidence.

Security Observatory is the bounded case study for the delivery method, not an AI-enabled product claim. The case study covers delivery across a private Salesforce product repository, a public website repository and private governance. The principal claim is narrow: AI-assisted delivery can be more governable when authority, evidence, baseline identity and uncertainty are represented explicitly.

Contents

1. Introduction

1.1 The capacity problem in small software projects

A small software project is not merely a large project with fewer people. The same individual may define requirements, design the architecture, implement the change, review the code, write the tests, update documentation and decide whether to release. This concentration produces speed and continuity, but removes many natural challenge mechanisms.

Security-sensitive software sharpens the problem. A small change can alter authorisation, evidence retention, data exposure, deployment behaviour or the meaning of a customer-facing statement. A system that reports security posture must protect execution integrity and epistemic integrity: the maintained correspondence among what the system observed, what it retained, what the project verified and what any public artefact asserts.

AI assistance can restore missing capacity, but capacity is not accountability. A model does not bear the consequence of an insecure release, a misleading claim or a corrupted repository. It cannot inherit the owner's duty to decide what is acceptable.

How can a small project use AI intensively while keeping intent, evidence, accountability and release authority human-owned?

1.2 The failure of informal AI assistance

Informal AI use often means describing an outcome, accepting a plausible implementation, running a few tests and moving on. Security-sensitive work requires explicit answers to five questions:

1. Was the requested change actually authorised?
2. Did implementation remain within security and data boundaries?
3. What evidence demonstrates the claimed behaviour?
4. Was the evidence challenged from a separate role, or merely restated?
5. Who accepted residual risk and authorised release?

Without those answers, a project can produce polished but weakly governed work. Common failures include requirement invention, scope laundering, evidence inflation, review convergence and false completion.

1.3 Purpose and contribution

This paper contributes a practical governance method rather than a new model architecture or universal software-development lifecycle. It combines AI risk management, secure software development, human oversight and evidence-bounded release governance in a lightweight operating loop. The method works with ordinary issue trackers, Git repositories, automated tests, static validation, web research and human review.

2. Scope, definitions and methodological stance

2.1 Scope

The method is intended for small software products and reference implementations where one person or a small team owns architecture and delivery; AI assists with research, drafting, coding, testing or review; security, privacy or customer-trust claims matter; repository history and release artefacts exist; and human release authority can be made explicit.

It is not a substitute for formal safety engineering, regulated validation, professional penetration testing, legal advice or independent certification.

2.2 Definitions

Terms used in this publication

Term	Meaning
AI-assisted delivery	Use of a generative model or coding agent in research, design, implementation, test, documentation or review.
Human authority	The retained human power to define scope, approve exceptions, accept risk, make public claims and authorise release.
Task contract	A bounded statement of authorised work, exclusions, starting state, evidence requirements and stop conditions.
Evidence gate	A condition satisfied by inspectable evidence before work advances to the next delivery state.
Role-separated adversarial challenge	Review from a distinct delivery role, model/provider perspective, prompt or evidence set. It is a challenge mechanism, not third-party assurance.
Baseline identity	The repository, worktree, branch, expected commit and clean or dirty state under review.
Adjudication	Human resolution of disagreement, uncertainty or conflicting evidence.
Release evidence	Evidence tied to the exact candidate or deployed artefact.
Governance close-out	Updating requirements, issue state and retained evidence after live verification.

2.3 Methodological stance

The method is evidence-bounded: source inspection, automated test, runtime observation and live deployment checks remain distinct evidence classes. It is human-governed: authority is embedded where intent, security boundaries, public claims and release risk are decided. It is proportionate: control concentrates at task authorisation, sensitive data, adversarial challenge, release evidence and public claims.

3. Risk model for AI-assisted delivery

3.1 Risk is introduced at the interaction boundary

Risk emerges from interaction among the model, prompt, tools, repository state, external services and human expectations. The relevant unit of control is the authorised interaction: what the model may see, what it may change, what evidence it must produce and what decisions remain outside its authority.

3.2 Principal failure modes

Requirement and scope

Requirement invention fills missing authority with statistical expectation. **Scope laundering** introduces product changes as tidy-up or best practice.

Evidence and review

False completion ignores deployment or visual acceptance. **Evidence inflation** promotes source or fixture observations. **Review correlation** mistakes repeated framing for confirmation.

Security and operation

Generated weaknesses, sensitive-data disclosure, automation bias, context collision and operator fatigue can redirect correct work into the wrong authority or evidence boundary.

Language models can identify superficial problems in their own output, but reliable reasoning correction generally requires external feedback.[10] Empirical work has also found security weaknesses in generated code in public repositories.[9] Domain experts can accept improper automated suggestions and take less initiative,[11] while explanations do not reliably reduce automation bias and can increase it.[12]

3.3 Risk is not reduced by model plurality alone

Several models can improve challenge coverage, but plurality is not assurance unless roles and evidence are separated. Different providers may still share public material, reasoning patterns or the same mistaken project framing. The human adjudicator resolves results against source, tests, runtime evidence or authoritative external material.

4. Governance principles

1. **Human-owned intent.** AI may propose clarification, but ambiguity does not become authority.
2. **Bounded authority.** State what may be read and changed, what remains untouched and when work stops.
3. **Source hierarchy.** Prefer current owner decisions, canonical requirements, current source and tests, runtime evidence and then historical material; model inference and conversational memory come last.
4. **Evidence before acceptance.** A narrative is not proof. Use source paths, diffs, tests, deployment results, HTTP responses, rendered pages, checksums or owner acceptance records.
5. **Role separation.** Separate implementation, challenge and release authority even when one human owns the project.
6. **Data minimisation.** Exclude secrets, tokens, session identifiers, keys, certificates, customer evidence and raw sensitive diagnostics unless explicitly authorised.

7. **Traceable state transitions.** Work advances because required evidence exists, not merely because activity occurred.
8. **Reproducibility.** Identify repository, worktree, branch, baseline, status, changed files, checks, resulting commit and artefact.
9. **Consensus is not proof.** Agreement is a review signal; evidence remains the truth criterion.
10. **Public claims are release artefacts.** Customer wording, diagrams and papers receive the same evidence discipline as code.
11. **Operator state is part of the boundary.** When role, tree or baseline cannot be stated without guessing, return to read-only work or stop.

5. The evidence-gated delivery cycle

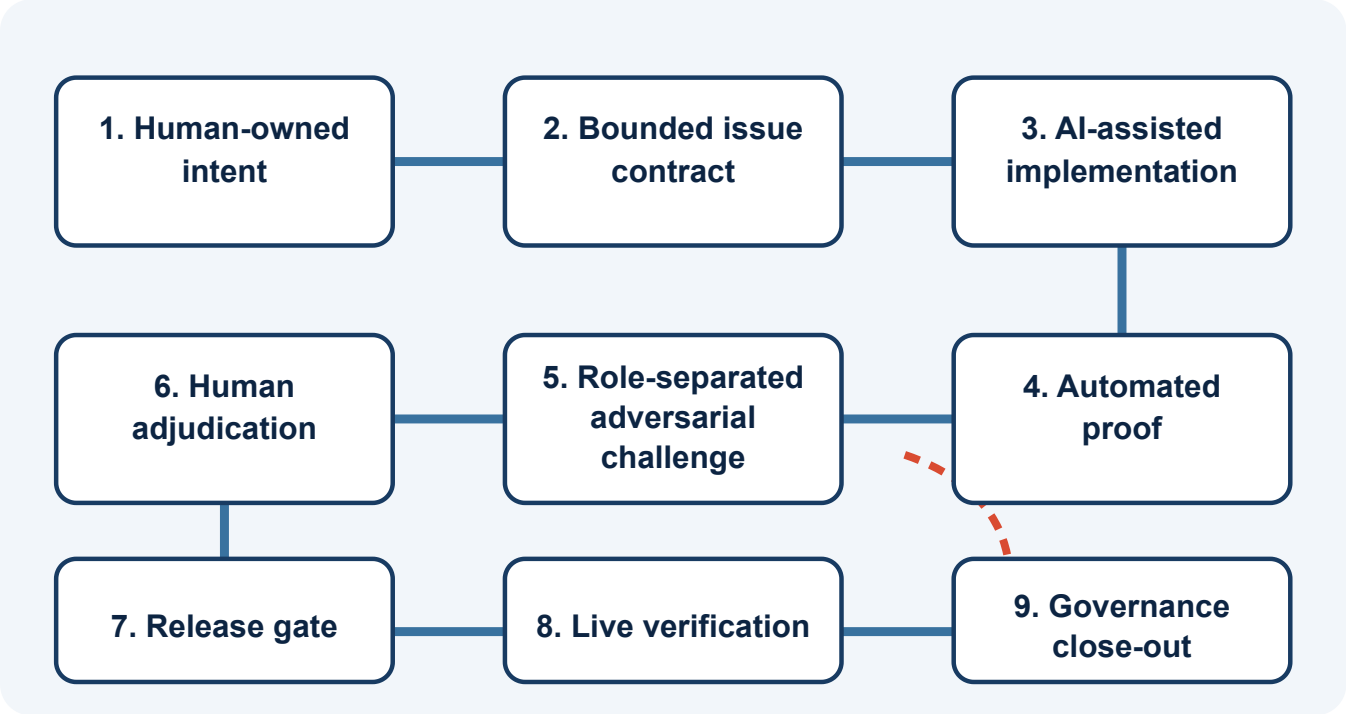

Figure 1. Each transition is gated by the minimum evidence for the target state; challenge and adjudication may return work to an earlier state.

1. **Human-owned intent and constraints.** Minimum evidence: owner decision or canonical requirement.
2. **Bounded issue contract.** Minimum evidence: a specification tied to the current baseline.
3. **AI-assisted implementation.** Minimum evidence: source diff and changed-file inventory.
4. **Automated proof.** Minimum evidence: build, test and validation output tied to the candidate, including negative checks where relevant.
5. **Role-separated adversarial challenge.** Minimum evidence: findings with source references and explicit uncertainty.
6. **Human adjudication.** Minimum evidence: owner decision and any updated task boundary.
7. **Release gate.** Minimum evidence: release checklist tied to a candidate SHA or immutable artefact identity.
8. **Live verification.** Minimum evidence: live observation plus artefact identity.

9. **Governance close-out.** Minimum evidence: updated issue/status and retained release record.

The sequence is not a rigid waterfall. Feedback may return work to an earlier state, but each transition remains explicit and evidence-gated.

6. Roles and separation of responsibility

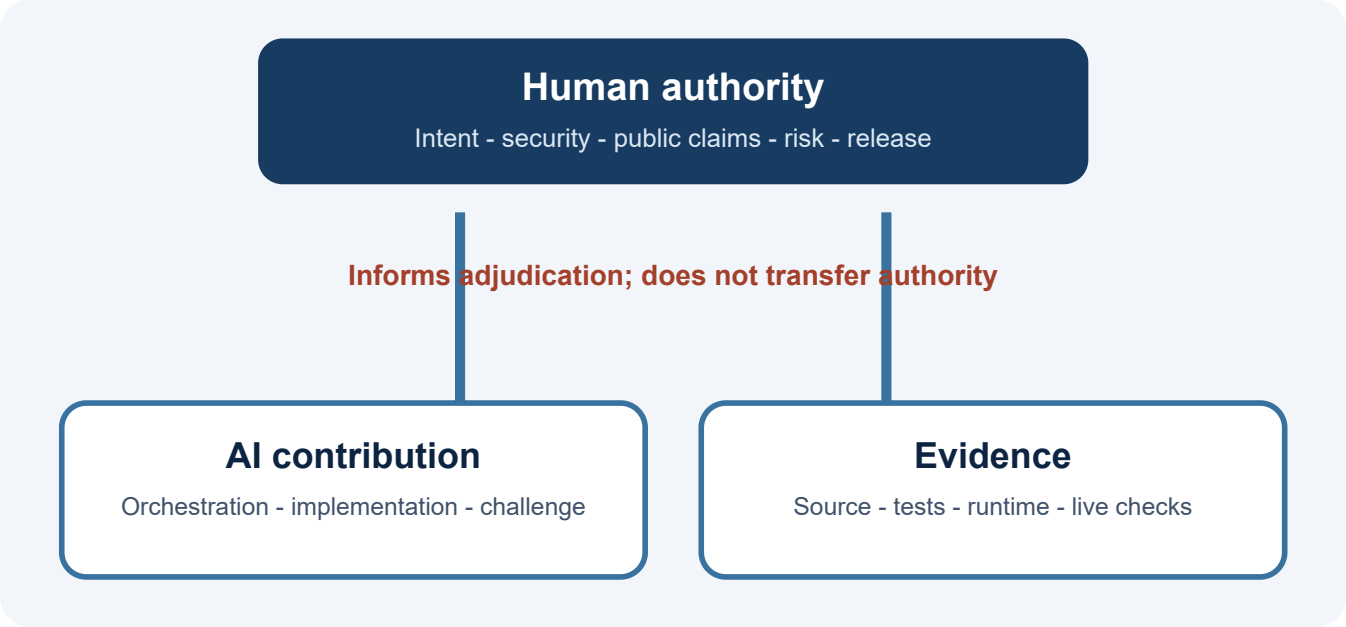

Figure 2. Human authority, AI contribution and evidence remain separate.

6.1 Owner, architect and release authority

The human owner controls intent, architecture, security boundaries, risk acceptance, legal and public wording, visual acceptance and release. In the case study this role was held by Luca Pacini.

6.2 Orchestration, synthesis and research

A general-purpose model can translate decisions into bounded tasks, compare findings, research primary sources and maintain a view across repositories. ChatGPT/GPT primarily served this role. Current state was re-read from canonical requirements, GitHub or the live web when it mattered.

6.3 Implementation

The implementation role converts a bounded requirement into source changes, tests and validation evidence. Codex primarily served this role against explicitly identified product and website baselines. A successful implementation report did not constitute deployment proof.

6.4 Role-separated adversarial challenge

Claude and Claude Code were used to challenge technical, customer-facing and publication claims. This increased the chance of finding different errors, but did not create professional third-party assurance.

6.5 Automated and external evidence

Tests, validators, build checks, checksums and HTTP checks supplied reproducible observations. Official sources and the public web supplied external evidence. Each source proved only the condition actually exercised.

6.6 Replaceable tools, durable authority rules

The method is vendor-neutral even though the case study names its tools. What must survive tool changes is separation among authority, contribution, challenge and evidence.

7. The task contract

A task contract reduces ambiguity before work begins and makes review objective by comparing the candidate with an authorised specification rather than inferred intention.

Required task-contract elements

Element	Required content
Requirement anchor	Issue, requirement or owner-decision identifier.
Repository identity	Repository, worktree and source boundary.
Starting baseline	Branch, expected commit and clean or dirty condition.
Objective	The smallest complete outcome.
In scope	Authorised files, behaviours and artefacts.
Out of scope	Prohibited changes and deferred work.
Security boundaries	Secrets, data, authorisation and remediation restrictions.
Evidence requirements	Tests, validators, runtime checks and artefact integrity.
Stop conditions	Wrong repository, unrelated changes, missing source, divergent baseline or absent authority.
Publication rules	Commit, push, review, release and live-verification gates.

Exclusions matter because models tend to complete patterns. A removed card may be replaced because a layout looks incomplete; a newly available slot may acquire an unverified claim. A stop condition protects the project from operating on the wrong state and should preserve that state while reporting the blocker.

8. Evidence classes and permitted claims

Delivery evidence ladder

Evidence class	Meaning	Permitted claim
Proposed	A change has been suggested.	May be considered; not implemented.

Evidence class	Meaning	Permitted claim
Source-established	Behaviour is visible in reviewed source.	Implemented in that source; runtime is not implied.
Test-observed	A controlled automated test exercised behaviour.	Observed under the stated fixture.
Runtime-observed	Behaviour occurred in a reviewed execution.	Observed in that execution and environment.
Release-validated	The exact candidate passed defined release gates.	Validated for that candidate and prerequisites.
Live-verified	The deployed artefact was checked after publication.	Present at the checked live boundary.
Supported	The project accepts behaviour in its product contract.	Supported within declared scope and prerequisites.
Expected	Required by design but not evidenced higher.	Expectation or release-gate requirement only.
No conclusion available	No reliable project-level evidence exists.	No positive or negative conclusion.

Evidence cannot be silently promoted. Source does not become release validation because it looks straightforward. A local build does not prove a hosted page is live. One organisation does not establish universal compatibility.

The ladder classifies what the delivery project may claim about its work. It is not subscriber-facing Security Observatory vocabulary. The companion Evidence Semantics and Scanner Orchestration paper defines that separate presentation model.

9. Security and data boundaries

9.1 Least information to the model

Default prohibited context includes access and refresh tokens, session identifiers, client secrets, passwords, keys, certificate bodies, raw sensitive headers, production customer evidence, unnecessary personal data and attacker-useful diagnostics.

9.2 Repository and context boundaries

Agents receive only the repository, worktree and evidence required for the task. Product source, website source and governance remain distinct. Cross-tree reasoning is permitted only where required; cross-tree editing is not assumed.

9.3 No autonomous remediation

An AI-assisted task may not introduce write-back actions, token revocation, permission removal, API blocking or endpoint modification into a product whose contract excludes them. Such changes require explicit owner authorisation, threat modelling and separate release evidence.

9.4 Prompt and transcript retention

Assume prompts and tool output may persist. Exclude sensitive values before interaction rather than relying on later deletion.

9.5 External research and the live-web boundary

Use authoritative sources for platform, security, standards and legal-adjacent claims. Search snippets identify leads, not load-bearing evidence. Live website observations prove only the checked public boundary; source inspection does not prove a cache, index or hosted page has updated.

---

10. Role-separated adversarial challenge and human adjudication

10.1 Challenger objective

A challenger tries to falsify claims: scope, baseline identity, prohibited states, test relevance, local-to-live promotion, customer wording and security boundaries.

10.2 Challenge without scope capture

Classify each finding as accepted defect, already tracked, deferred improvement, unsupported assertion, regression risk, new owner decision or leave alone. This prevents review from replacing the roadmap.

10.3 Provider separation is not third-party assurance

Another model may reduce direct self-review correlation but can share sources, assumptions or framing. Reserve the word independent for genuinely separate external assurance or replication.

10.4 Resolving disagreement

Decompose disagreement into factual propositions and check current owner decisions, source, tests, runtime evidence or authoritative material. Where evidence remains incomplete, leave the result unresolved.

10.5 Human visual and professional judgement

Layout balance, tone, diagram readability and customer credibility are not wholly reducible to automation. A manual visual gate is legitimate evidence when bounded to defined surfaces and recorded separately.

---

11. Case study: Security Observatory

> **Case-study boundary.** Security Observatory is used as a bounded software-delivery case study. It is not described as having AI product capabilities. AI tools contributed only to the engineering workflow under human authority.

11.1 Project context

Security Observatory is a Salesforce-native security evidence application with read-only assessment and no security remediation or write-back to assessed Salesforce configuration. Its delivery combines Salesforce architecture, security design, package engineering, evidence semantics, documentation, a public website, licensing and release governance.

11.2 Actual tool and authority allocation

Case-study role allocation

Delivery role	Implementation	Authority boundary
Owner, architect, adjudicator and release authority	Luca Pacini	Owns scope, security boundaries, wording, risk acceptance, visual acceptance and release.
Orchestration, synthesis and research	ChatGPT/GPT	Structures tasks, compares evidence and researches sources; it is not repository truth or release authority.
Bounded implementation and source inspection	Codex	Edits authorised source, creates tests and reports evidence against a named baseline.
Role-separated adversarial challenge	Claude and Claude Code	Challenges technical and public claims; findings require human adjudication.
Automated evidence	Tests, builds, validators, checksums and HTTP checks	Proves only the exercised condition on the exact candidate or checked surface.

11.3 Requirements and governance as the control plane

Numbered requirements, bounded GitHub issues, explicit statuses and retained owner decisions became authoritative state. Chat history and model memory remained convenient but lossy. Current canonical and repository state were re-read before implementation, review or status reporting.

11.4 Multi-repository and multi-worktree orchestration

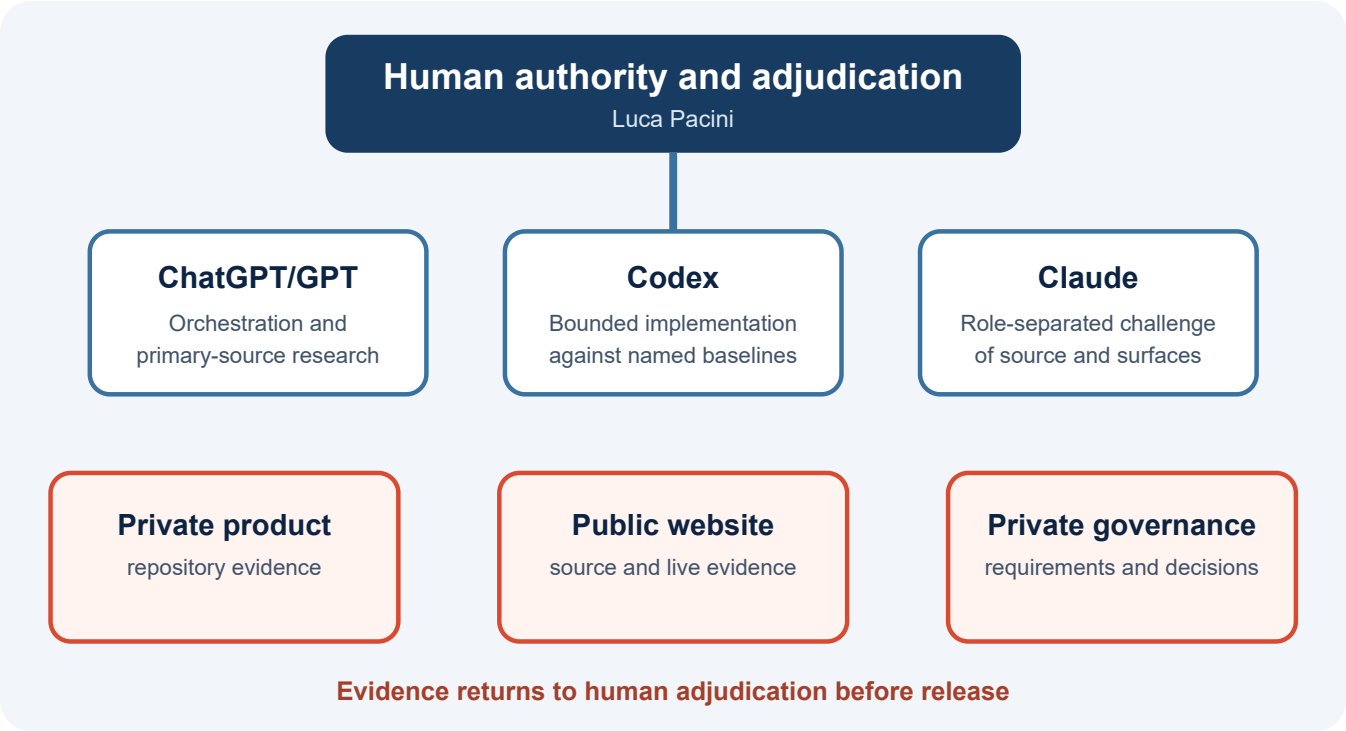

Figure 3. Case-study orchestration across three evidence domains.

Repository identity was treated as an evidence boundary. Before material work, the expected repository, worktree, branch, HEAD and clean or dirty state were established. A model can reason perfectly about the wrong branch and still produce a useless answer. Departures from the expected tree required an explicit reason, commit identity and statement of included and excluded work.

11.5 Product delivery path

1. Owner defines the security and customer outcome.
2. Governance records requirements and exclusions.
3. Codex changes only the authorised baseline.
4. Tests and validators supply the required evidence class.
5. Claude or Claude Code challenges implementation and claims.
6. ChatGPT/GPT helps decompose disagreement and verify external sources.
7. The owner adjudicates and retains release authority.
8. Deployment, package or runtime proof remains a separate gate.

11.6 Public website and publication path

1. Owner decision and issue.
2. Public-site baseline verification.
3. Bounded source change.
4. Static build and content/security checks.
5. Source and customer-surface challenge.
6. Human adjudication and visual acceptance.

7. Publication.
8. Live URL and artefact identity checks.
9. Owner-controlled indexing where relevant.
10. Close-out only after the live boundary matches the candidate.

11.7 Separate evidence domains

Private source supports implementation and baseline identity, not live publication. Automated validation supports the exercised condition, not visual quality or universal support. Official sources support external claims, not private implementation. Public web checks support what a reader could retrieve at that time, not its private build history. Human adjudication supports accepted scope and risk, not otherwise unproved technical behaviour.

11.8 Complete trace: evidence-first website release

A completed website trace followed the full loop: approved public outcome; bounded source and validation rules; implementation on the current baseline; copy, navigation, metadata and prohibited-language checks; role-separated challenge; human visual acceptance; publication; live verification; and governance close-out.

11.9 Non-confirming product cases

Four product failures demonstrated the value and limits of challenge. EV-01 and EV-02 were false-zero paths after source acquisition failed. EV-03 lost a useful bounded cause from an exception. EV-04 could collapse a bounded retained-evidence read failure into an unexplained Unknown presentation. The method did not prevent the defects; challenge caught them and stopped favourable claims until the evidence changed.

11.10 Publication-process failures and hardening

Publication work exposed wrong or stale worktree assumptions, loss of editable source, wording broader than test evidence, reviewer scope capture, insufficient automated visual proof, out-of-sequence artefact and landing-page changes, and drift from current product vocabulary. Controls were hardened with baseline preflight, controlled source reconstruction, narrow claims, finding classification, explicit visual acceptance and separate artefact, link and live checks.

11.11 Human factors

Several similar model chats, browser tabs, terminals and worktrees made operator context switching part of the threat model. The discipline became: name role and tree; check root, branch, HEAD and status; keep one tree write-enabled; label copied material by destination; and stop publication or writes when fatigue makes context uncertain.

11.12 Human-only decisions

- Scope freezes and priority.
- Security exceptions or product-contract changes.
- Acceptance of reviewer recommendations.
- Legal and public wording.
- Visual acceptance.
- Package and release readiness.
- Publication and residual-risk acceptance.

11.13 Context withheld from AI systems

The project did not deliberately supply secrets, access or refresh tokens, session identifiers, private keys, certificate bodies, raw customer evidence, attacker-useful diagnostics or autonomous remediation authority.

11.14 What the case study does not prove

The case study does not prove greater speed, generalisation to larger teams, independence from model diversity or sufficiency of a human gate. It shows only that one technically experienced owner used AI intensively in an engineering workflow while keeping authority, evidence classes and release gates explicit.

---

12. Measuring value without productivity theatre

12.1 Productivity evidence is context-dependent

Published evidence on AI-assisted programming varies by population, task, tool generation, outcome measure and date. Cui and colleagues report about 26 per cent more completed tasks across three field experiments.[14] METR reported that sampled experienced maintainers using early-2025 tools took about 19 per cent longer,[15] then cautioned in 2026 that newer-tool estimates were only weak evidence because of selection and measurement problems.[24] Song, Agarwal and Wen report project-level contribution and participation effects alongside increased coordination time.[16]

These studies are not commensurable. A small project should avoid a universal headline figure and instead ask whether work passes evidence gates and release authority within an accepted residual-risk boundary.

12.2 Recommended measures

Proposed instrumentation; no values are reported in this paper

Measure	What it tests	Misuse to avoid
Issue-to-live lead time	Delivery throughput	Ignoring rework or review debt.
Rework after challenge	First-pass quality and task clarity	Treating all challenge-driven change as failure.
Scope deviation count	Task-contract effectiveness	Rewarding under-delivery.
Escaped defect count	Release effectiveness	Comparing releases of different risk.
Evidence-gate failures	Whether controls are active	Treating blockers as wasted time.
Human review time	Governance cost	Assuming less is always better.
Unsupported-claim corrections	Epistemic quality	Hiding corrections to improve the metric.
Sensitive-data incidents	Boundary effectiveness	Ignoring near misses.
Live verification failures	Local-to-live fidelity	Omitting infrastructure factors.

12.3 Quality-adjusted capacity

The useful outcome is not maximum generated code. It is correctly scoped, reviewable and releasable work completed without increasing residual risk beyond the accepted boundary.

13. Anti-patterns

1. **The giant prompt.** Product design, implementation, documentation, deployment and review lose boundaries.
2. **The self-certifying agent.** One agent implements, tests, interprets and declares readiness.
3. **Model voting.** Majority agreement replaces evidence.
4. **Test-green absolutism.** Passing available tests becomes proof of requirements or public claims.
5. **Validator weakening.** A guard is relaxed merely because new work fails it.
6. **Unbounded tidy-up.** Unrelated refactors or additions enter a narrow issue.
7. **Secret-rich context.** Sanitisation happens after disclosure.
8. **Publication before evidence.** Claims precede exact-candidate validation.
9. **Artificial freshness.** Dates or status change without substantive work.
10. **Endless review churn.** Every reviewer preference becomes scope.
11. **The six-window cockpit.** Similar chats and terminals rely on memory for role and tree, destroying provenance.

14. Proportionate adoption for a small project

14.1 Minimum viable control set

1. Canonical requirements.
2. Bounded issues with exclusions and stop conditions.
3. Protected source with identifiable baselines.
4. Automated build, test and validation.
5. Role-separated challenge record.
6. Release and live-verification record.

14.2 Risk-based tailoring

Low-risk documentation may need a source diff, link checks and owner review. Changes to authorisation, sensitive data, package installation or public security claims require stronger evidence and exact-candidate validation.

14.3 When the method should become heavier

Expand governance when customer or production data is processed, software changes customer security configuration, regulation applies, multiple teams contribute, deployment is autonomous or failure exceeds the owner's ability to absorb it.

15. Relationship to established guidance

15.1 What the cited instruments govern

NIST AI RMF and its Generative AI Profile address organisational AI risk.[1][2] NIST SSDF addresses secure software development, and SP 800-218A extends that guidance to AI model development and systems incorporating models.[3][4] UK secure-AI and AI cyber-security guidance treat security across the AI lifecycle.[5][6] The Software Security Code of Practice sets voluntary software-security principles,[8] and UK assurance guidance frames evidence for trustworthy claims.[7]

15.2 Adjacent bodies of practice

Goal Structuring Notation and the NCSC Assurance Principles and Claims express claims, arguments and evidence.[19][25] SLSA and in-toto address supply-chain provenance,[20][21] while this method establishes the repository, worktree, branch and commit actually used before work begins. ISO/IEC 42001 defines organisational AI-management requirements.[22] OWASP GenAI guidance covers threats in systems embedding generative AI; this paper covers a model as an engineering contributor whether or not the delivered system contains AI.[23]

15.3 The bounded gap

Within the reviewed set, the instruments do not specify the same lightweight loop combining repository baseline identity, exclusions, evidence classes, role-separated challenge, human adjudication and live/publication verification for a small team. This is not a claim that no comparable practice exists or that the method is a new standard.

16. Limitations and research agenda

16.1 Single-project derivation

The method comes from one project with a technically experienced owner and may depend on architectural skill and security judgement.

16.2 The assessor is not independent

Luca Pacini designed the method, is its only practitioner to date and adjudicated every disputed finding. Favourable project-specific statements are self-attested unless a public artefact or external source is identified. Stronger standing requires independent replication.

16.3 No controlled productivity comparison

No controlled non-AI baseline exists, and the proposed measures have no reported values. The paper cannot quantify speed, cost or defect reduction attributable to AI assistance.

16.4 Model and tool evolution

Capabilities, context windows, controls and data-governance terms change quickly. Express controls at role and evidence level, not permanently by product.

16.5 Human oversight is fallible

Human overseers can fail to correct model error and can introduce bias.[12][13] The case study's fatigue and wrong-context episodes show that operator state belongs in the threat model. A human gate is necessary, not sufficient.

16.6 Private evidence

Some case-study evidence remains in private repositories. Public readers cannot reproduce every source-established statement.

16.7 Future research

- Controlled comparison with informal AI-assisted delivery.
- Measurement of review correlation.
- Task-contract patterns by risk class.
- Proof of context sanitisation.
- Machine-readable evidence registers.
- Assurance of autonomous tool use.
- Relationship between human expertise and challenge quality.

---

17. Practical adoption checklist

Before the task

- Is the owner decision explicit?
- Are repository, worktree, branch, expected HEAD and status known?
- Are model, browser and terminal windows labelled by role and tree?
- Is the operator alert enough for write-capable work?
- Are inclusions, exclusions, sensitive-data boundaries, stop conditions and evidence requirements stated?

During implementation

- Is only authorised source changing?
- Does each command sequence still run in the expected root, branch, HEAD and state?
- Is the instruction going to the intended role?
- Are claims tied to evidence and validators strengthened rather than bypassed?
- Are blockers reported while repository state is preserved?

Before release

- Did automated proof pass on the exact candidate?
- Did challenge use the correct baseline?
- Did the owner adjudicate?
- Did visual or customer-facing acceptance occur?
- Do public statements stay within evidence?

- Are artefact identity and checksums recorded where relevant?

After release

- Was the live boundary checked?
- Does the deployed artefact match the candidate?
- Were requirements and issue state updated?
- Are residual limitations retained?
- Were stale sessions closed?
- Was monitoring assigned without representing it as completed implementation?

---

18. Conclusion

AI can expand the work a small software project can attempt only while delivery authority remains distinct from delivery capacity. A model may generate code, tests, prose, research and critique. It cannot inherit responsibility for scope, security, truth or release.

Evidence-gated delivery treats every transition as a claim requiring proof. Task contracts define authority and exclusions. Baseline identity identifies the software under review. Automated checks provide reproducible evidence. Role-separated challenge exposes assumptions without becoming third-party assurance. Human adjudication resolves uncertainty. Release and live verification establish exact-artefact state. Governance close-out preserves what remains unproved.

The Security Observatory case study adds a practical observation: multi-model orchestration is useful only when multi-tree provenance is explicit. Security Observatory remains the bounded case study and is not represented as an AI-capable product.

> **A small project can use AI intensively without allowing fluency, model consensus or automation success to replace human authority and evidence.**

Appendix A. Reusable bounded task contract

Requirement anchor

State the authoritative issue, requirement or owner decision.

Repository and baseline

- Repository
- Worktree
- Branch
- Expected starting commit or HEAD
- Active role and window label
- Write-enabled or read-only
- Required clean or dirty condition
- Permitted committed-HEAD-only exception

Objective

State the smallest complete outcome.

In scope

List authorised behaviours, files and artefacts.

Out of scope

List prohibited changes, deferred items and adjacent work.

Security and data boundaries

State prohibited secrets, data classes, authorisation changes, remediation actions and external egress.

Validation

List exact build, test, static, security, visual, package, runtime and live-verification requirements.

Stop conditions

State when the agent must stop without modification.

Commit and publication

State commit rules, push authority, release gate and live-verification sequence.

Final report

Require starting and ending state, changed-file tree, validation evidence, unresolved risks and final repository status.

Appendix B. Minimal delivery evidence register

Template only. No completed entries are published here; the case-study delivery evidence register is retained privately.

Fields: Claim; Scope; Evidence class; Artefact; Limitation; Next gate; Owner decision.

References

1. NIST, Artificial Intelligence Risk Management Framework (AI RMF 1.0), 2023.
2. NIST, Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, 2024.
3. NIST, Secure Software Development Framework (SSDF) Version 1.1, 2022.
4. NIST, Secure Software Development Practices for Generative AI and Dual-Use Foundation Models, 2024.
5. National Cyber Security Centre, Guidelines for Secure AI System Development, 2023.
6. Department for Science, Innovation and Technology, AI Cyber Security Code of Practice, 2025.
7. Department for Science, Innovation and Technology, Introduction to AI Assurance, 2024.
8. Department for Science, Innovation and Technology and NCSC, Software Security Code of Practice, 2025.
9. Fu et al., Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study, 2025.
10. Huang et al., Large Language Models Cannot Self-Correct Reasoning Yet, ICLR 2024.
11. Levy et al., Assessing the Impact of Automated Suggestions on Decision Making, 2021.
12. Vered et al., The Effects of Explanations on Automation Bias, 2023.
13. Green and Chen, The Principles and Limits of Algorithm-in-the-Loop Decision Making, 2019.
14. Cui et al., The Effects of Generative AI on High-Skilled Work, 2026.
15. METR, Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity, 2025.
16. Song, Agarwal and Wen, The Impact of Generative AI on Collaborative Open-Source Software Development, revised 2026.
17. Jimenez et al., SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, ICLR 2024.
18. Fortmilo, Evidence Semantics and Scanner Orchestration, 2026.
19. Assurance Case Working Group, Goal Structuring Notation Community Standard, Version 3, 2021.
20. OpenSSF, Supply-chain Levels for Software Artifacts (SLSA), Version 1.2, 2025.
21. Torres-Arias et al., in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes, 2019.
22. ISO/IEC, ISO/IEC 42001:2023 Artificial intelligence management system, 2023.
23. OWASP GenAI Security Project, OWASP Top 10 for LLM Applications 2025.
24. METR, We are Changing our Developer Productivity Experiment Design, 2026.

25. National Cyber Security Centre, Software Security Code of Practice - Assurance Principles and Claims, 2025.

Publication and licensing

> **Version 1.0 accessible edition, 30 August 2026 - Current.** Regenerated at the stable publication path with semantic tagging, document language, bookmarks, linked references, figure alternatives and embedded fonts. The previous PDF remains available through Git history only.

Author: Luca Pacini. **Publisher:** Fortmilo. **Public location:** United Kingdom.

Competing interests. The author is the founder and sole proprietor of Fortmilo. Security Observatory is the bounded case-study product, and this paper forms part of the Fortmilo publication surface. No external funding was received.

AI-use disclosure. Generative AI systems contributed research, synthesis, implementation in case-study repositories, document production and role-separated challenge under the authority model described here. The author adjudicated outputs and retains sole responsibility for the published claims.

This paper is not a certification, industry standard, third-party assurance report or universal productivity claim. It is a proposed engineering method derived from one bounded case study and the cited guidance and research.

Stable current URL: https://fortmilo.co.uk/documents/orchestrating-ai-for-secure-software-delivery.pdf